

决策 Transformer: 通过序列建模进行强化学习

Lili Chen^{*,1}, Kevin Lu^{*,1}, Aravind Rajeswaran², Kimin Lee¹,
Aditya Grover², Michael Laskin¹, Pieter Abbeel¹, Aravind Srinivas^{†,1}, Igor Mordatch^{†,3}

* 同等贡献

† 同等指导

¹UC Berkeley ²Facebook AI Research ³Google Brain

{lilichen, kz1}@berkeley.edu

摘要

本文提出一个框架，将强化学习（Reinforcement Learning, RL）抽象为序列建模问题。这使得我们能够利用 Transformer 架构的简洁性与可扩展性，以及语言建模领域的相关进展，如 GPT-x 和 BERT。具体而言，我们提出决策 Transformer（Decision Transformer），一种将强化学习问题转化为条件序列建模的架构。与以往拟合价值函数或计算策略梯度的强化学习方法不同，决策 Transformer 通过利用因果掩码 Transformer 直接输出最优动作。通过以期望回报（奖励）、过去状态和动作为条件训练自回归模型，我们的决策 Transformer 模型能够生成实现期望回报的未来动作。尽管结构简单，决策 Transformer 在 Atari、OpenAI Gym 和 Key-to-Door 任务上达到或超越了最先进的无模型离线强化学习基线的性能。

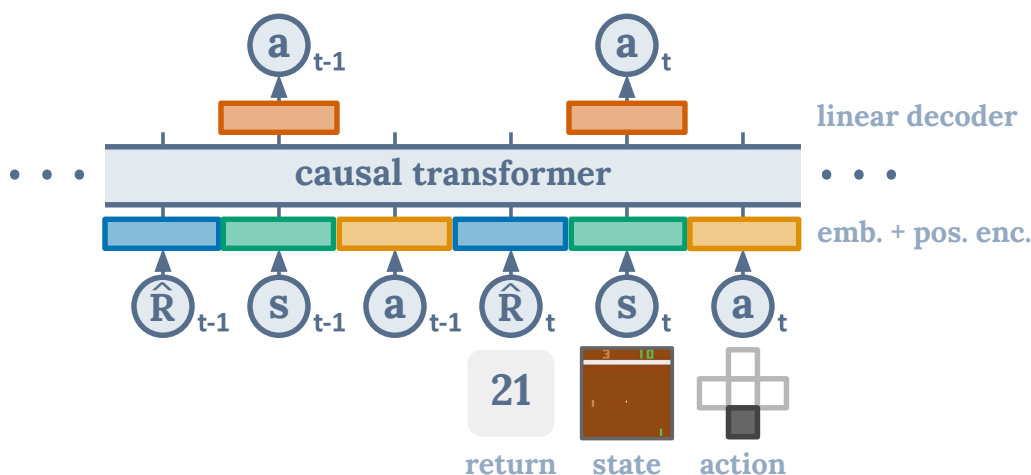


图 1: 决策 Transformer 架构 1。状态、动作和回报被输入到模态特定的线性嵌入中，并添加位置性的回合时间步编码。令牌被输入到 GPT 架构中，该架构使用因果自注意力掩码自回归地预测动作。

¹Our code is available at: <https://sites.google.com/berkeley.edu/decision-transformer>

目录

1 引言 3

2 预备知识 4

2.1 离线强化学习	4
2.2 变换器	4

3 方法 4

4 离线强化学习基准上的评估 6

4.1 雅达利	6
4.2 开放人工智能健身房	7

5 讨论 8

5.1 决策 Transformer 是否在数据子集上执行行为克隆?	8
5.2 决策 Transformer 对回报分布的建模效果如何?	8
5.3 使用更长上下文长度的好处是什么?	9
5.4 决策 Transformer 是否执行有效的长期信用分配?	9
5.5 变换器能否在稀疏奖励环境中成为准确的评论家?	10
5.6 决策 Transformer 在稀疏奖励环境中表现良好吗?	10
5.7 为什么决策 Transformer 避免了价值悲观或行为正则化的需要?	10
5.8 决策 Transformer 如何有益于在线强化学习机制?	11

6 相关工作 11

6.1 离线强化学习	11
6.2 强化学习环境中的监督学习	11
6.3 信用分配	12
6.4 条件语言生成	12
6.5 注意力与变换器模型	12

7 结论 12

A 实验细节 18

A.1 Atari	18
A.2 OpenAI Gym	18
A.2.1 Decision Transformer	18
A.2.2 Behavior Cloning	19
A.3 Graph Shortest Path	19

B Atari Task Scores 20

1 引言

近期研究表明, 变换器 (Transformer) [1] 能够在大规模上对语义概念的高维分布进行建模, 包括在语言 [2] 中实现有效的零样本泛化, 以及在图像生成 [3] 中实现分布外泛化。鉴于此类模型在多种应用中的成功, 本文旨在探究其应用于形式化为强化学习 (Reinforcement Learning, RL) 的序列决策问题。与先前将变换器作为传统强化学习算法 [4, 5] 中组件的架构选择不同, 本文研究生成式轨迹建模——即对状态、动作和奖励序列的联合分布进行建模——能否替代传统强化学习算法。

本文考虑以下范式转变：不再通过传统强化学习算法（如时序差分（Temporal Difference, TD）学习 [6]）训练策略，而是利用序列建模目标在收集的经验上训练变换器模型。这将使我们能够绕过用于长期信用分配的自举（bootstrapping）需求，从而避免已知会破坏强化学习稳定性的“致命三要素”[6]之一。同时，这也避免了时序差分学习中通常采用的未来奖励折扣，该做法可能诱发不良的短视行为。此外，本文可以利用语言和视觉领域广泛使用的现有变换器框架，这些框架易于扩展，并受益于大量关于变换器模型稳定训练的研究工作。

除了已展示的建模长序列的能力外，变换器（Transformer）还具有其他优势。变换器可以通过自注意力（Self-Attention）直接进行信用分配，这与贝尔曼备份（Bellman Backup）缓慢传播奖励且易受“干扰”信号影响形成对比 [7]。这使得变换器在奖励稀疏或存在干扰的情况下仍能有效工作。最后，经验证据表明，变换器建模方法能够对广泛的行为分布进行建模，从而实现更好的泛化和迁移 [3]。

我们通过考虑离线强化学习 (Offline RL) 来探索这一假设, 在该任务中, 智能体需要从次优数据中学习策略——从固定且有限的经验中产生最大有效的行为。由于误差传播和价值高估, 该任务传统上具有挑战性 [8]。然而, 当使用序列建模目标进行训练时, 这是一个自然的任务。通过在状态、动作和回报序列上训练自回归模型, 我们将策略采样简化为自回归生成建模。我们可以通过选择期望的回报标记来指定策略的专业水平——即查询哪种“技能”, 这些标记充当生成的提示。

示例说明。为了直观理解我们的提议，考虑在有向图上寻找最短路径的任务，该任务可以表述为强化学习问题。当智能体位于目标节点时奖励为 0，否则为 -1 。我们训练一个 GPT[9] 模型来预测序列中的下一个标记，该序列由回报至目标（未来奖励之和）、状态和动作组成。仅在随机游走数据上训练——没有专家演示——我们可以在测试时通过添加先验来生成尽可能高的回报（更多细节和实证结果见附录），随后通过条件生成相应的动作序列，从而生成最优轨迹。因此，通过将序列建模工具与事后回报信息相结合，我们实现了策略改进，而无需动态规划。

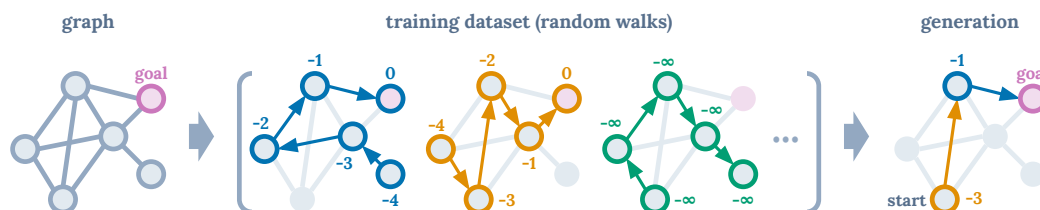


图 2: 在固定图 (左) 上寻找最短路径的示例, 该问题被表述为强化学习。训练数据集由随机游走轨迹及其每个节点的回报至目标 (中) 组成。以起始状态为条件, 并在每个节点生成尽可能大的回报, 决策变换器 (Decision Transformer) 生成最优路径序列。

受此观察启发，本文提出决策 Transformer (Decision Transformer)，利用 GPT 架构对轨迹进行自回归建模（如图 1 所示）。通过在 Atari [10]、OpenAI Gym [11] 和 Key-to-Door [12] 环境中的离线强化学习基准上评估决策 Transformer，研究序列建模能否执行策略优化。结果表明，在不使用动态规划的情况下，决策 Transformer 的性能达到或超过了最先进的无模型离线强化学习算法 [13, 14]。此外，在需要长期信用分配的任务中，决策 Transformer 显著优于强化学习基线。通过这项工作，本文旨在将序列建模和 Transformer 与强化学习联系起来，并希望序列建模成为强化学习的一种强大算法范式。

2 预备知识

2.1 离线强化学习

本文考虑在由元组 $(\mathcal{S}, \mathcal{A}, P, \mathcal{R})$ 描述的马尔可夫决策过程 (Markov Decision Process, MDP) 中进行学习。该 MDP 元组包含状态 $s \in \mathcal{S}$ 、动作 $a \in \mathcal{A}$ 、转移动态 $P(s'|s, a)$ 以及奖励函数 $r = \mathcal{R}(s, a)$ 。本文使用 s_t, a_t 和 $r_t = \mathcal{R}(s_t, a_t)$ 分别表示时间步 t 处的状态、动作和奖励。轨迹由一系列状态、动作和奖励组成： $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T, a_T, r_T)$ 。时间步 $t, R_t = \sum_{t'=t}^T$ 处轨迹的回报 $r_{t'}$ ，是从该时间步起未来奖励的总和。强化学习的目标是学习一个策略，以最大化 MDP 中的期望回报 $\mathbb{E}[\sum_{t=1}^T r_t]$ 。在离线强化学习中，我们无法通过与环境交互获取数据，只能访问由任意策略的轨迹展开组成的固定有限数据集。该设置更具挑战性，因为它剥夺了智能体探索环境并收集额外反馈的能力。

2.2 Transformer

Transformer 由 Vaswani 等人 [1] 提出，作为一种高效建模序列数据的架构。这些模型由堆叠的自注意力层和残差连接组成。每个自注意力层接收对应于唯一输入标记的 n 个嵌入 $\{x_i\}_{i=1}^n$ ，并输出 n 个嵌入 $\{z_i\}_{i=1}^n$ ，保持输入维度不变。第 i 个标记通过线性变换映射为键 k_i 、查询 q_i 和值 v_i 。自注意力层的第 i 个输出通过查询 q_i 与其他键 k_j 之间的归一化点积对值 v_j 进行加权得到：

$$z_i = \sum_{j=1}^n \text{softmax}(\{\langle q_i, k_{j'} \rangle\}_{j'=1}^n)_j \cdot v_j. \quad (1)$$

正如我们稍后将看到的，这允许该层通过查询向量和键向量的相似性（最大化点积）隐式地形成状态-回报关联来分配“信用”。在这项工作中，我们使用 GPT 架构 [9]，它通过因果自注意力掩码修改了变换器架构以实现自回归生成，将 n 个标记上的求和/Softmax 函数替换为仅对序列中先前标记的求和/Softmax 函数 ($j \in [1, i]$)。我们将其他架构细节留给原始论文。

3 方法

在本节中，我们提出决策 Transformer，它以对变换器架构的最小修改来自回归地建模轨迹，如图 1 和算法 1 所总结。

轨迹表示。我们选择轨迹表示的关键要求是，它应使变换器能够学习有意义的模式，并且我们能够在测试时有条件地生成动作。对奖励进行建模并非易事，因为我们希望模型基于未来期望回报而非过去奖励来生成动作。因此，我们不是直接输入奖励，而是向模型提供回报到目标 $R_t = \sum_{t'=t}^T r_{t'}$ 。这导致了以下适合自回归训练和生成的轨迹表示：

$$\tau = (\hat{R}_1, s_1, a_1, \hat{R}_2, s_2, a_2, \dots, \hat{R}_T, s_T, a_T). \quad (2)$$

在测试时，我们可以指定期望的性能（例如，成功为 1，失败为 0）以及环境起始状态，作为启动生成的条件信息。在为当前状态执行生成的动作后，我们将目标回报减去已获得的奖励，并重复直到回合终止。

架构。 我们将最后 K 个时间步输入决策 Transformer，总共 $3K$ 个标记（每个模态一个：回报到目标、状态或动作）。为了获得标记嵌入，我们为每个模态学习一个线性层，将原始输入投影到嵌入维度，然后进行层归一化 [15]。对于具有视觉输入的环境，状态被输入到卷积编码器而不是线性层。此外，为每个时间步学习一个嵌入并添加到每个标记——注意这与变换器使用的标准位置嵌入不同，因为一个时间步对应三个标记。然后，这些标记由 GPT [9] 模型处理，该模型通过自回归建模预测未来的动作标记。

训练。 给定一个离线轨迹数据集。我们从数据集中采样序列长度为 K 的小批量。与输入标记 s_t 对应的预测头被训练来预测 a_t ——对于离散动作使用交叉熵损失，对于连续动作使用均方误差——并且每个时间步的损失被平均。我们发现预测状态或回报目标并不能提升性能，尽管这在我们框架内很容易实现（如第 5.4 节所示），并且将是未来工作的一个有趣研究。

Algorithm 1 Decision Transformer Pseudocode (for continuous actions)

```
# R, s, a, t: returns-to-go, states, actions, or timesteps
# transformer: transformer with causal masking (GPT)
# embed_s, embed_a, embed_R: linear embedding layers
# embed_t: learned episode positional embedding
# pred_a: linear action prediction layer

# main model
def DecisionTransformer(R, s, a, t):
    # compute embeddings for tokens
    pos_embedding = embed_t(t) # per-timestep (note: not per-token)
    s_embedding = embed_s(s) + pos_embedding
    a_embedding = embed_a(a) + pos_embedding
    R_embedding = embed_R(R) + pos_embedding

    # interleave tokens as (R_1, s_1, a_1, ..., R_K, s_K)
    input_embeddings = stack(R_embedding, s_embedding, a_embedding)

    # use transformer to get hidden states
    hidden_states = transformer(input_embeddings=input_embeddings)

    # select hidden states for action prediction tokens
    a_hidden = unstack(hidden_states).actions

    # predict action
    return pred_a(a_hidden)

# training loop
for (R, s, a, t) in dataloader: # dims: (batch_size, K, dim)
    a_preds = DecisionTransformer(R, s, a, t)
    loss = mean((a_preds - a)**2) # L2 loss for continuous actions
    optimizer.zero_grad(); loss.backward(); optimizer.step()

# evaluation loop
target_return = 1 # for instance, expert-level return
R, s, a, t, done = [target_return], [env.reset()], [], [1], False
while not done: # autoregressive generation/sampling
    # sample next action
    action = DecisionTransformer(R, s, a, t)[-1] # for cts actions
    new_s, r, done, _ = env.step(action)

    # append new tokens to sequence
    R = R + [R[-1] - r] # decrement returns-to-go with reward
    s, a, t = s + [new_s], a + [action], t + [len(R)]
    R, s, a, t = R[-K:], ... # only keep context length of K
```

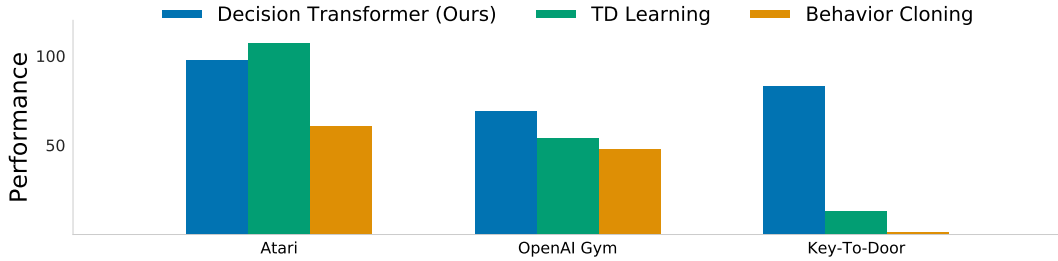


图 3: 比较决策 Transformer (我们的方法) 与 TD 学习 (CQL) 和行为克隆在 Atari、OpenAI Gym 和 Minigrid 上的结果。在多样化的任务集上, 决策 Transformer 的表现与传统方法相当或更好。性能通过归一化回合回报来衡量 (详见正文)。

4 离线强化学习基准上的评估

在本节中, 我们研究决策 Transformer 相对于专用离线强化学习和模仿学习算法的性能。特别是, 我们的主要比较对象是基于 TD 学习的无模型离线强化学习算法, 因为我们的决策 Transformer 架构本质上也是无模型的。此外, TD 学习是强化学习中样本效率的主导范式, 并且在许多基于模型的强化学习算法中作为子程序突出出现 [16, 17]。我们还与行为克隆及其变体进行比较, 因为它也涉及与我们类似的基于似然的策略学习公式。具体算法取决于环境, 但我们的动机如下:

- **TD 学习:** 这些方法大多使用动作空间约束或价值悲观主义, 将是对决策 Transformer 最忠实的比较, 代表标准强化学习方法。最先进的无模型方法是保守 Q 学习 (CQL) [14], 作为我们的主要比较对象。此外, 我们还与其他先前的无模型强化学习算法如 BEAR[18] 和 BRAC[19] 进行比较。

- **模仿学习:** 该机制同样采用监督损失进行训练, 而非贝尔曼备份。本文在此使用行为克隆, 并在第 5.1 节中给出更详细的讨论。

本文在离散 (Atari [10]) 和连续 (OpenAI Gym [11]) 控制任务上进行了评估。前者涉及高维观测空间, 并要求长期信用分配; 后者则需要精细的连续控制, 代表了一组多样化的任务。主要结果汇总于图 3, 其中展示了每个领域的平均归一化性能。

4.1 Atari

Atari 基准 [10] 因其高维视觉输入以及动作与结果奖励之间的延迟所导致的信用分配困难而具有挑战性。本文按照 Agarwal 等人 [13] 的方法, 在 DQN 重放数据集中所有样本的 1% 上评估所提方法, 这相当于在线 DQN 智能体 [20] 在训练期间观察到的 5000 万次转移中的 50 万次; 报告 3 个随机种子的均值和标准差。本文根据 Hafner 等人 [21] 的协议, 基于专业玩家对分数进行归一化, 其中 100 代表专业玩家分数, 0 代表随机策略。

本文在 Agarwal 等人 [13] 评估的四个 Atari 任务 (Breakout、Qbert、Pong 和 Seaquest) 上与 CQL[14]、REM[13] 和 QR-DQN[22] 进行了比较。对于决策 Transformer, 本文使用上下文长度为 $K = 30$ (Pong 除外, 为 $K = 50$)。本文还报告了行为克隆的性能, 该方法使用与决策 Transformer 相同的网络架构和超参数, 但没有回报条件²。对于 CQL、REM 和 QR-DQN 基线, 本文直接引用 CQL 和 REM 论文中的数值。结果如表 1 所示。所提方法在 4 个游戏中的 3 个上与 CQL 具有竞争力, 并在所有 4 个游戏上优于或持平于 REM、QR-DQN 和行为克隆。

² 本文还尝试了如先前工作那样使用具有 $K = 1$ 的多层感知机, 但发现其性能不如 Transformer。

Game	DT (Ours)	CQL	QR-DQN	REM	BC
Breakout	267.5 $\pm$ 97.5	211.1	17.1	8.9	138.9 $\pm$ 61.7
Qbert	15.4 $\pm$ 11.4	104.2	0.0	0.0	17.3 $\pm$ 14.7
Pong	106.1 $\pm$ 8.1	111.9	18.0	0.5	85.2 $\pm$ 20.0
Seaquest	2.5 $\pm$ 0.4	1.7	0.4	0.7	2.1 $\pm$ 0.3

表 1: 1% DQN 重放 Atari 数据集上的玩家归一化分数。本文报告 3 个随机种子的均值和方差。最佳均值分数以粗体突出显示。决策 Transformer 在 4 个游戏中的 3 个上与 CQL 表现相当，并在大多数游戏中优于其他基线。

4.2 OpenAI Gym

在本节中，我们考虑 D4RL 基准 [23] 中的连续控制任务。我们还考虑了一个不属于该基准的二维机械臂环境，并使用与 D4RL 基准类似的方法生成数据集。机械臂是一个目标条件任务，奖励稀疏，因此它代表了与标准运动环境（半猎豹、跳跃者、步行者）不同的设置。不同的数据集设置描述如下。

1. 中等：由“中等”策略生成的 100 万时间步，该策略达到专家策略约三分之一的得分。
2. 中等回放：训练到中等策略性能的智能体的回放缓冲区（在我们的环境中大约 25k-400k 时间步）。
3. 中等专家：由中等策略生成的 100 万时间步与由专家策略生成的 100 万时间步拼接而成。

我们与 CQL [14]、BEAR [18]、BRAC [19] 和 AWR [24] 进行比较。CQL 代表了无模型离线强化学习的最新水平，是带有价值悲观主义的时序差分学习的一种实例。得分按照 Fu 等人 [23] 的方法进行归一化，使 100 代表专家策略。CQL 数据来自原始论文；BC 数据由我们运行；其他方法的数据来自 D4RL 论文。我们的结果如表 2 所示。决策 Transformer 在大多数任务中取得了最高得分，并在其余任务中与最新技术具有竞争力。

Dataset	Environment	DT (Ours)	CQL	BEAR	BRAC-v	AWR	BC
Medium-Expert	HalfCheetah	86.8 $\pm$ 1.3	62.4	53.4	41.9	52.7	59.9
Medium-Expert	Hopper	107.6 $\pm$ 1.8	111.0	96.3	0.8	27.1	79.6
Medium-Expert	Walker	108.1 $\pm$ 0.2	98.7	40.1	81.6	53.8	36.6
Medium-Expert	Reacher	89.1 $\pm$ 1.3	30.6	-	-	-	73.3
Medium	HalfCheetah	42.6 $\pm$ 0.1	44.4	41.7	46.3	37.4	43.1
Medium	Hopper	67.6 $\pm$ 1.0	58.0	52.1	31.1	35.9	63.9
Medium	Walker	74.0 $\pm$ 1.4	79.2	59.1	81.1	17.4	77.3
Medium	Reacher	51.2 $\pm$ 3.4	26.0	-	-	-	48.9
Medium-Replay	HalfCheetah	36.6 $\pm$ 0.8	46.2	38.6	47.7	40.3	4.3
Medium-Replay	Hopper	82.7 $\pm$ 7.0	48.6	33.7	0.6	28.4	27.6
Medium-Replay	Walker	66.6 $\pm$ 3.0	26.7	19.2	0.9	15.5	36.9
Medium-Replay	Reacher	18.0 $\pm$ 2.4	19.0	-	-	-	5.4
Average (Without Reacher)		74.7	63.9	48.2	36.9	34.3	46.4
Average (All Settings)		69.2	54.2	-	-	-	47.7

表 2: D4RL 数据集 3 的结果。我们报告了三个种子的均值和方差。决策 Transformer (DT) 在几乎所有任务上都优于传统强化学习算法。

³ 鉴于 CQL 通常是最强的时序差分学习方法，对于机械臂环境，我们仅运行 CQL 基线。

5 讨论

5.1 决策 Transformer 是否在数据子集上执行行为克隆？

在本节中，我们试图深入了解决策 Transformer 是否可以被视为在具有特定回报的数据子集上执行模仿学习。为此，我们提出了一种新方法，百分位行为克隆（%BC），其中我们仅对数据集中按回合回报排序的前 $X\%$ 时间步执行行为克隆。百分位 $X\%$ 在标准行为克隆（ $X = 100\%$ ）和仅克隆最佳观测轨迹（ $X \rightarrow 0\%$ ）之间进行插值，在通过更多数据训练以获得更好泛化与训练专注于理想数据子集的专用模型之间进行权衡。

我们在表 3 中展示了 %BC 与决策 Transformer 和 CQL 的完整比较结果，扫描 $X \in [10\%, 25\%, 40\%, 100\%]$ 了。请注意，选择用于克隆的最优子集的唯一方法是使用环境中的回滚进行评估，因此 %BC 并非一种现实可行的方法；相反，它旨在提供对决策 Transformer 行为的洞见。当数据充足时——如在 D4RL 环境中——我们发现 %BC 可以匹敌或超越其他离线强化学习方法。在大多数环境中，决策

Transformer 与最佳 %BC 的性能相当，表明它可以在整个数据集分布上训练后聚焦于特定子集。

Dataset	Environment	DT (Ours)	10%BC	25%BC	40%BC	100%BC	CQL
Medium	HalfCheetah	42.6 $\pm$ 0.1	42.9	43.0	43.1	43.1	44.4
Medium	Hopper	67.6 $\pm$ 1.0	65.9	65.2	65.3	63.9	58.0
Medium	Walker	74.0 $\pm$ 1.4	78.8	80.9	78.8	77.3	79.2
Medium	Reacher	51.2 $\pm$ 3.4	51.0	48.9	58.2	58.4	26.0
Medium-Replay	HalfCheetah	36.6 $\pm$ 0.8	40.8	40.9	41.1	4.3	46.2
Medium-Replay	Hopper	82.7 $\pm$ 7.0	70.6	58.6	31.0	27.6	48.6
Medium-Replay	Walker	66.6 $\pm$ 3.0	70.4	67.8	67.2	36.9	26.7
Medium-Replay	Reacher	18.0 $\pm$ 2.4	33.1	16.2	10.7	5.4	19.0
Average		56.1	56.7	52.7	49.4	39.5	43.5

表 3: 决策 Transformer (DT) 与百分位行为克隆 (%BC) 的比较。

相比之下，当我们研究低数据场景时——例如 Atari，我们使用重放缓冲区的 1% 作为数据集——%BC 表现较弱（如表 4 所示）。这表明在数据量相对较少的情况下，决策 Transformer 可以通过利用数据集中的所有轨迹来改善泛化，从而超越 %BC，即使这些轨迹与回报条件目标不相似。我们的结果表明，决策 Transformer 比简单地在数据子集上执行模仿学习更有效。在我们考虑的任务上，决策 Transformer 要么优于 %BC，要么与之相当，且无需选择最优子集的困扰。

Game	DT (Ours)	10%BC	25%BC	40%BC	100%BC
Breakout	267.5 $\pm$ 97.5	28.5 $\pm$ 8.2	73.5 $\pm$ 6.4	108.2 $\pm$ 67.5	138.9 $\pm$ 61.7
Qbert	15.4 $\pm$ 11.4	6.6 $\pm$ 1.7	16.0 $\pm$ 13.8	11.8 $\pm$ 5.8	17.3 $\pm$ 14.7
Pong	106.1 $\pm$ 8.1	2.5 $\pm$ 0.2	13.3 $\pm$ 2.7	72.7 $\pm$ 13.3	85.2 $\pm$ 20.0
Seaquest	2.5 $\pm$ 0.4	1.1 $\pm$ 0.2	1.1 $\pm$ 0.2	1.6 $\pm$ 0.4	2.1 $\pm$ 0.3

表 4: Atari 上的 %BC 得分。我们报告了 3 个随机种子的均值和方差。决策 Transformer (DT) 在大多数游戏中优于所有版本的 %BC。

5.2 决策 Transformer 对回报分布的建模效果如何？

我们通过在大范围内改变期望目标回报来评估决策 Transformer 理解回报目标标记的能力，从而评估变换器的多任务分布建模能力。图 4 展示了在评估回合中，智能体在不同目标回报值下累积的平均采样回报。在所有任务上，期望目标回报与真实观测回报高度相关。在 Pong、HalfCheetah 和 Walker 等任务上，决策 Transformer 生成的轨迹几乎完美匹配期望回报

（如与理想线的重叠所示）。此外，在 Seaquest 等一些 Atari 任务上，我们可以用高于数据集中最大回合回报的回报来提示决策 Transformer，这表明决策 Transformer 有时能够进行外推。

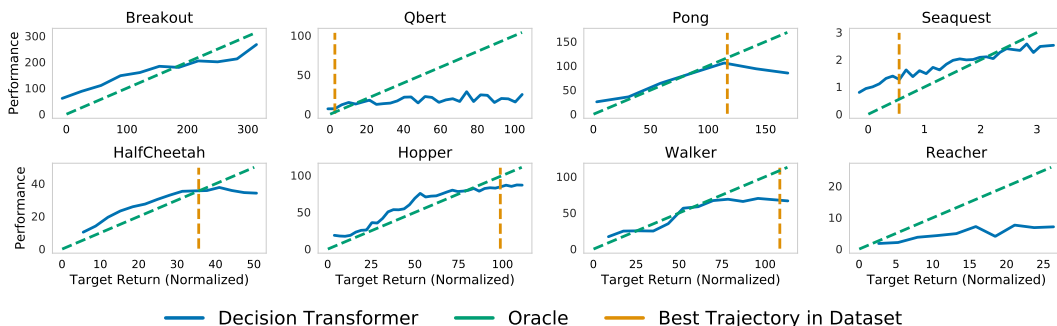


图 4：决策 Transformer 在指定目标（期望）回报条件下累积的采样（评估）回报。上：Atari。下：D4RL 中等重放数据集。

5.3 使用更长上下文长度有何益处？

为了评估访问先前状态、动作和回报的重要性，我们对上下文长度 K 进行了消融。这一点值得关注，因为通常认为在使用帧堆叠时，先前状态（即 $K = 1$ ）对于强化学习算法已经足够，正如我们所做的那样。表 5 显示，当 $K = 1$ 时，决策 Transformer 的性能显著下降，这表明过去的信息对 Atari 游戏是有用的。一个假设是，当我们表示策略分布时——例如通过序列建模——上下文使变换器能够识别哪个策略生成了动作，从而实现更好的学习和/或改善训练动态。

Game	DT (Ours)	DT with no context ($K = 1$)
Breakout	267.5 $\pm$ 97.5	73.9 $\pm$ 10
Qbert	15.1 $\pm$ 11.4	13.6 $\pm$ 11.3
Pong	106.1 $\pm$ 8.1	2.5 $\pm$ 0.2
Seaquest	2.5 $\pm$ 0.4	0.6 $\pm$ 0.1

表 5：上下文长度消融。决策 Transformer (DT) 在使用更长上下文长度时表现更好（Pong 为 $K = 50$ ，其他为 $K = 30$ ）。

5.4 决策 Transformer 能否进行有效的长期信用分配？

为评估模型在长期信用分配方面的能力，我们考虑 Mesnard 等人提出的“钥匙到门”环境的一个变体 [12]。这是一个基于网格的环境，包含三个阶段序列：（1）在第一阶段，智能体被放置在一个有钥匙的房间中；（2）随后，智能体被放置在一个空房间中；（3）最后，智能体被放置在一个有门的房间中。智能体在第三阶段到达门时获得二元奖励，但前提是它在第一阶段拾取了钥匙。该问题对信用分配具有挑战性，因为信用必须从回合开始传播到结束，跳过中间采取的动作。

我们在由随机动作生成的轨迹数据集上进行训练，并在表 6 中报告成功率。此外，对于钥匙开门环境，我们使用整个回合长度作为上下文，而不是像其他环境那样使用固定的内容窗口。使用高视界回报信息的方法：我们的决策 Transformer 模型和 %BC（仅在成功回合上训练）能够学习有效的策略——尽管仅在随机游走上训练，却能产生接近最优的路径。时序差分学习（CQL）无法在涉及的长视界上有效传播 Q 值，因此性能较差。

Dataset	DT (Ours)	CQL	BC	%BC	Random
1K Random Trajectories	71.8%	13.1%	1.4%	69.9%	3.1%
10K Random Trajectories	94.6%	13.3%	1.6%	95.1%	3.1%

表 6: 钥匙开门环境下的成功率。采用事后经验的方法（决策 Transformer、%BC）能够学习到成功的策略，而时序差分学习则难以进行信用分配。

5.5 变换器能否在稀疏奖励环境中成为准确的评判者？

在前几节中，我们确立了决策 Transformer 能够产生有效策略（执行者）。我们现在评估变换器模型是否也能成为有效的评论者。我们修改决策 Transformer，使其在钥匙到门环境中除动作令牌外还输出回报令牌。此外，第一个回报令牌不是给定的，而是被预测的（即模型学习初始分布 $p(\hat{R}_1)$ ），类似于标准的自回归生成模型。我们发现变换器根据回合中的事件持续更新奖励概率，如图所示图 5（左）。此外，本文发现变换器（Transformer）关注情节中的关键事件（拾取钥匙或到达门口），如图 5（右）所示，这表明状态-奖励关联的形成，正如 Raposo 等人 [25] 所讨论的，并实现了准确的价值预测。

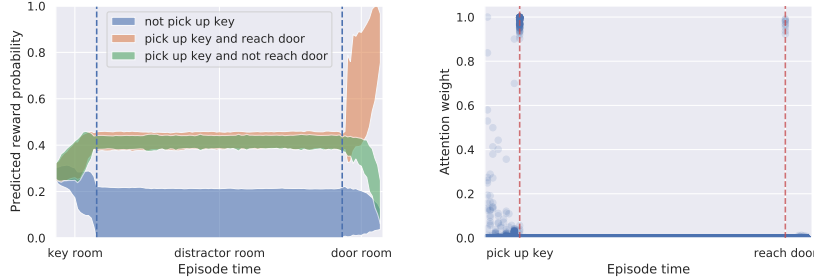


图 5: 左图: 变换器模型对三种回合结果预测的运行回报概率平均值。右图: 某个成功回合中所有时间步的变换器注意力权重叠加图。模型关注回合中关键事件附近的步骤，例如拾取钥匙和到达门口。

5.6 决策变换器在稀疏奖励设置下表现良好吗？

时序差分学习算法的一个已知弱点是，为了表现良好，它们需要密集的奖励，这可能不现实且成本高昂。相比之下，决策变换器可以在这些设置下提高鲁棒性，因为它对奖励密度的假设极少。为了评估这一点，我们考虑了 D4RL 基准的延迟回报版本，其中智能体在轨迹过程中不接收任何奖励，而是在最后时间步接收轨迹的累积奖励。我们针对延迟回报的结果如表 7 所示。延迟回报对决策变换器的影响极小；并且由于训练过程的性质，模仿学习方法对奖励不敏感。当时序差分学习崩溃时，决策变换器和 %BC 仍然表现良好，这表明决策变换器对延迟奖励更具鲁棒性。

Dataset	Environment	Delayed (Sparse)		Agnostic		Original (Dense)	
		DT (Ours)	CQL	BC	%BC	DT (Ours)	CQL
Medium-Expert	Hopper	107.3 ± 3.5	9.0	59.9	102.6	107.6	111.0
Medium	Hopper	60.7 ± 4.5	5.2	63.9	65.9	67.6	58.0
Medium-Replay	Hopper	78.5 ± 3.7	2.0	27.6	70.6	82.7	48.6

表 7: 具有延迟（稀疏）奖励的 D4RL 数据集结果。决策变换器（DT）和模仿学习受密集奖励移除的影响极小，而 CQL 失败。

5.7 为什么决策变换器避免了价值悲观或行为正则化的需要？

决策变换器与先前离线强化学习算法的一个关键区别在于，我们不需要策略正则化或保守性即可实现良好性能。我们的猜想是，基于时序差分学习的算法学习一个近似价值函数，并通过优化该价值函数来改进策略。这种优化已学习函数的行为会加剧并利用价值函数近似中的任何不准确性，从而导致策略改进失败。由于决策变换器不需要使用已学习函数作为目标进行显式优化，因此它避免了正则化或保守性的需要。

5.8 决策变换器如何有益于在线强化学习机制？

离线强化学习（Offline RL）以及建模行为的能力，有潜力为下游任务实现样本高效的在线强化学习。研究从离线到在线过渡的工作普遍发现，基于似然的方法（如我们的序列建模目标）更为成功 [26, 27]。因此，尽管本文研究的是离线强化学习，我们相信决策 Transformer（Decision Transformer）能够作为行为生成的强模型，显著改进在线强化学习方法。例如，决策 Transformer 可以充当强大的“记忆引擎”，并与 Go-Explore 等强大的探索算法结合 [28]，有潜力同时建模并生成多样化的行为集合。

6 相关工作

6.1 离线强化学习

为缓解离线强化学习中的分布偏移（Distribution Shift）影响，先前算法要么（a）约束策略动作空间 [29, 30, 31]，要么（b）引入价值悲观主义 [29, 14]，要么（c）将悲观主义纳入学习到的动力学模型 [32, 33]。由于本文未使用决策 Transformer 显式学习动力学模型，因此主要与无模型算法进行比较；特别地，加入动力学模型往往会提升无模型算法的性能。另一条研究路线通过从离线数据集中学习任务无关的技能集来探索广泛的行为分布，方法包括基于似然的方法 [34, 35, 36, 37] 或最大化互信息 [38, 39, 40]。本文工作与基于似然的方法相似，这些方法不使用迭代贝尔曼更新——尽管我们采用更简单的序列建模目标而非变分方法，并利用奖励进行行为的条件生成。

6.2 强化学习环境中的监督学习

一些先前的强化学习方法更类似于静态监督学习，例如 Q 学习 [41, 42]，它仍然使用迭代备份，或者基于似然的方法如行为克隆，它们不使用（在前一节中讨论过）。最近的工作 [43, 44, 45] 研究了“颠倒”强化学习（UDRL），这与我们的方法相似，都试图用以目标回报为条件的监督损失来建模行为。我们工作的一个关键区别是将动机从监督学习转向序列建模：虽然实际方法主要在上下文长度和架构上有所不同，但序列建模即使在没有奖励的情况下也能进行行为建模，类似于语言 [9] 或图像 [46] 的风格，并且已知具有良好的扩展性 [2]。Kumar 等人 [44] 提出的方法与我们的方法最相似，使用 $K = 1$ ，我们发现序列建模/长上下文表现更优（见第 5.3 节）。Ghosh 等人 [47] 扩展了先前的 UDRL 方法，使用状态目标条件而不是奖励，Paster 等人 [48] 进一步使用带有状态目标条件的 LSTM 用于目标条件在线强化学习设置。

与我们的工作同时，Janner 等人 [49] 提出了轨迹 Transformer，它与决策 Transformer 类似，但额外使用了状态和回报预测以及离散化，这包含了基于模型的组件。我们相信，他们的实验以及我们的结果，突显了序列建模作为强化学习中普遍适用思想的潜力。

6.3 信用分配

许多工作通过状态关联研究了更好的信用分配，学习一种分解奖励函数的架构，使得某些“重要”状态构成大部分信用 [50, 51, 12]。他们使用学习到的奖励函数来改变演员-评论家算法的奖励，以帮助在长时间跨度的传播信号。特别是，与我们的长期设置类似，一些工作专门表明这种状态关联架构在延迟奖励设置中表现更好 [52, 7, 53, 25]。相比之下，我们允许这些特性在 Transformer 架构中自然涌现，而无需显式学习奖励函数或评论家。

6.4 条件语言生成

多项工作研究了图像 [54] 和语言 [55, 56] 的引导生成。若干工作 [57, 58, 59, 60, 61, 62] 探索了可控文本生成模型的训练或微调。类条件语言模型也可用于学习判别器以引导生成 [63, 55, 64, 65]。然而，这些方法大多假设“类别”恒定不变，而在强化学习中奖励信号是时变的。此外，更自然的方式是向模型提示期望的目标回报，并随时间根据观测到的奖励持续递减该目标，因为变换器模型与环境共同生成轨迹。

6.5 注意力与变换器模型

变换器 (Transformer) [1] 已成功应用于自然语言处理 [66, 9] 和计算机视觉 [67, 68] 的众多任务。然而，变换器在强化学习中的研究相对较少，主要由于问题性质不同，例如训练方差较高。Zambaldi 等人 [5] 表明，通过关系推理增强变换器可提升组合环境中的性能；Ritter 等人 [69] 表明，迭代自注意力使强化学习智能体能够更好地利用情景记忆。Parisotto 等人 [4] 讨论了在高方差强化学习环境中更稳定训练变换器的设计决策。与本文工作不同，这些方法仍使用演员-评论家算法进行优化，侧重于架构的新颖性。此外，在模仿学习中，一些工作研究了将变换器作为长短期记忆网络的替代方案：Dasari 和 Gupta[70] 研究了单样本模仿学习，Abramson 等人 [71] 结合语言和图像模态进行文本条件行为生成。

7 结论

本文提出了决策变换器 (Decision Transformer)，旨在统一语言/序列建模与强化学习的思想。在标准离线强化学习基准上，我们证明了决策变换器仅需对标准语言建模架构进行最小修改，即可达到或超越专门为离线强化学习设计的强大算法的性能。

我们希望这项工作能激发更多关于将大型变换器模型用于强化学习的研究。我们在实验中采用了一种简单的监督损失，该损失在实验中表现有效，但应用于大规模数据集时，自监督预训练任务可能带来益处。此外，还可以考虑为回报、状态和动作设计更复杂的嵌入——例如，以回报分布为条件来建模随机环境，而非确定性回报。变换器模型也可用于建模轨迹的状态演化，有望成为基于模型的强化学习的替代方案，我们希望在未来的工作中探索这一方向。

对于实际应用而言，理解变换器在马尔可夫决策过程环境中产生的错误类型及其可能的负面后果至关重要，而这一领域尚未得到充分探索。同样重要的是考虑我们训练模型所用的数据集，这些数据集可能引入破坏性偏差，尤其是当我们考虑用更多可能来源存疑的数据来增强强化学习智能体时。例如，恶意行为者设计的奖励可能会在模型以期望回报为条件生成行为时引发非预期行为。

致谢

本研究得到了伯克利深度驾驶、开放慈善以及美国国家科学基金会（NSF:NRI #2024675）的支持。本工作部分完成于阿拉温德·拉杰斯瓦兰在华盛顿大学攻读博士期间，他当时获得了摩根大通人工智能博士奖学金（2020-21）的资助。我们还要感谢卢克·梅茨和丹尼尔·弗里曼提出的宝贵反馈与讨论，感谢贾斯汀·傅在搭建 D4RL 基准过程中的协助，以及阿维拉尔·库马尔在 CQL 基线和超参数方面的帮助。

参考文献

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.
- [2] Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*, 2020.
- [3] Aditya Ramesh, Mikhail Pavlov, Gabriel Goh, Scott Gray, Chelsea Voss, Alec Radford, Mark Chen, and Ilya Sutskever. Zero-shot text-to-image generation. *arXiv preprint arXiv:2102.12092*, 2021.
- [4] Emilio Parisotto, Francis Song, Jack Rae, Razvan Pascanu, Caglar Gulcehre, Siddhant Jayakumar, Max Jaderberg, Raphael Lopez Kaufman, Aidan Clark, Seb Noury, et al. Stabilizing transformers for reinforcement learning. In *International Conference on Machine Learning*, 2020.
- [5] Vinicius Zambaldi, David Raposo, Adam Santoro, Victor Bapst, Yujia Li, Igor Babuschkin, Karl Tuyls, David Reichert, Timothy Lillicrap, Edward Lockhart, et al. Deep reinforcement learning with relational inductive biases. In *International Conference on Learning Representations*, 2018.
- [6] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT Press, 2018.
- [7] Chia-Chun Hung, Timothy Lillicrap, Josh Abramson, Yan Wu, Mehdi Mirza, Federico Carnevale, Arun Ahuja, and Greg Wayne. Optimizing agent behavior over long time scales by transporting value. *Nature communications*, 10(1):1–12, 2019.
- [8] Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643*, 2020.
- [9] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. 2018.
- [10] Marc G Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, 2013.
- [11] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym. *arXiv preprint arXiv:1606.01540*, 2016.
- [12] Thomas Mesnard, Théophane Weber, Fabio Viola, Shantanu Thakoor, Alaa Saade, Anna Harutyunyan, Will Dabney, Tom Stepleton, Nicolas Heess, Arthur Guez, et al. Counterfactual credit assignment in model-free reinforcement learning. *arXiv preprint arXiv:2011.09464*, 2020.
- [13] Rishabh Agarwal, Dale Schuurmans, and Mohammad Norouzi. An optimistic perspective on offline reinforcement learning. In *International Conference on Machine Learning*, 2020.
- [14] Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative q-learning for offline reinforcement learning. In *Advances in Neural Information Processing Systems*, 2020.
- [15] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [16] Richard S. Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. In *ICML*, 1990.
- [17] Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine. When to trust your model: Model-based policy optimization. In *Advances in Neural Information Processing Systems*, pages 12498–12509, 2019.

- [18] Aviral Kumar, Justin Fu, George Tucker, and Sergey Levine. Stabilizing off-policy q-learning via bootstrapping error reduction. *arXiv preprint arXiv:1906.00949*, 2019.
- [19] Yifan Wu, George Tucker, and Ofir Nachum. Behavior regularized offline reinforcement learning. *arXiv preprint arXiv:1911.11361*, 2019.
- [20] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.
- [21] Danijar Hafner, Timothy Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering atari with discrete world models. *arXiv preprint arXiv:2010.02193*, 2020.
- [22] Will Dabney, Mark Rowland, Marc Bellemare, and Rémi Munos. Distributional reinforcement learning with quantile regression. In *Conference on Artificial Intelligence*, 2018.
- [23] Justin Fu, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. D4rl: Datasets for deep data-driven reinforcement learning. *arXiv preprint arXiv:2004.07219*, 2020.
- [24] Xue Bin Peng, Aviral Kumar, Grace Zhang, and Sergey Levine. Advantage-weighted regression: Simple and scalable off-policy reinforcement learning. *arXiv preprint arXiv:1910.00177*, 2019.
- [25] David Raposo, Sam Ritter, Adam Santoro, Greg Wayne, Theophane Weber, Matt Botvinick, Hado van Hasselt, and Francis Song. Synthetic returns for long-term credit assignment. *arXiv preprint arXiv:2102.12425*, 2021.
- [26] Yang Gao, Huazhe Xu, Ji Lin, Fisher Yu, Sergey Levine, and Trevor Darrell. Reinforcement learning from imperfect demonstrations. *arXiv preprint arXiv:1802.05313*, 2018.
- [27] Ashvin Nair, Murtaza Dalal, Abhishek Gupta, and Sergey Levine. Accelerating online reinforcement learning with offline datasets. *arXiv preprint arXiv:2006.09359*, 2020.
- [28] Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O Stanley, and Jeff Clune. Go-explore: a new approach for hard-exploration problems. *arXiv preprint arXiv:1901.10995*, 2019.
- [29] Scott Fujimoto, David Meger, and Doina Precup. Off-policy deep reinforcement learning without exploration. In *International Conference on Machine Learning*, 2019.
- [30] Aviral Kumar, Justin Fu, Matthew Soh, George Tucker, and Sergey Levine. Stabilizing off-policy q-learning via bootstrapping error reduction. In *Advances in Neural Information Processing Systems*, 2019.
- [31] Noah Y Siegel, Jost Tobias Springenberg, Felix Berkenkamp, Abbas Abdolmaleki, Michael Neunert, Thomas Lampe, Roland Hafner, and Martin Riedmiller. Keep doing what worked: Behavioral modelling priors for offline reinforcement learning. In *International Conference on Learning Representations*, 2020.
- [32] Rahul Kidambi, Aravind Rajeswaran, Praneeth Netrapalli, and Thorsten Joachims. Morel: Model-based offline reinforcement learning. In *Advances in Neural Information Processing Systems*, 2020.
- [33] Tianhe Yu, Garrett Thomas, Lantao Yu, Stefano Ermon, James Zou, Sergey Levine, Chelsea Finn, and Tengyu Ma. Mopo: Model-based offline policy optimization. In *Advances in Neural Information Processing Systems*, 2020.
- [34] Anurag Ajay, Aviral Kumar, Pulkit Agrawal, Sergey Levine, and Ofir Nachum. Opal: Offline primitive discovery for accelerating offline reinforcement learning. *arXiv preprint arXiv:2010.13611*, 2020.
- [35] Víctor Campos, Alexander Trott, Caiming Xiong, Richard Socher, Xavier Giro-i Nieto, and Jordi Torres. Explore, discover and learn: Unsupervised discovery of state-covering skills. In *International Conference on Machine Learning*, 2020.
- [36] Karl Pertsch, Youngwoon Lee, and Joseph J Lim. Accelerating reinforcement learning with learned skill priors. *arXiv preprint arXiv:2010.11944*, 2020.

- [37] Avi Singh, Huihan Liu, Gaoyue Zhou, Albert Yu, Nicholas Rhinehart, and Sergey Levine. Parrot: Data-driven behavioral priors for reinforcement learning. In *International Conference on Learning Representations*, 2021.
- [38] Benjamin Eysenbach, Abhishek Gupta, Julian Ibarz, and Sergey Levine. Diversity is all you need: Learning skills without a reward function. In *International Conference on Learning Representations*, 2019.
- [39] Kevin Lu, Aditya Grover, Pieter Abbeel, and Igor Mordatch. Reset-free lifelong learning with skill-space planning. *arXiv preprint arXiv:2012.03548*, 2020.
- [40] Archit Sharma, Shixiang Gu, Sergey Levine, Vikash Kumar, and Karol Hausman. Dynamics-aware unsupervised discovery of skills. In *International Conference on Learning Representations*, 2020.
- [41] Christopher Watkins. Learning from delayed rewards. 01 1989.
- [42] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.
- [43] Rupesh Kumar Srivastava, Pranav Shyam, Filipe Mutz, Wojciech Jaśkowski, and Jürgen Schmidhuber. Training agents using upside-down reinforcement learning. *arXiv preprint arXiv:1912.02877*, 2019.
- [44] Aviral Kumar, Xue Bin Peng, and Sergey Levine. Reward-conditioned policies. *arXiv preprint arXiv:1912.13465*, 2019.
- [45] Acting without rewards. 2019. URL <https://ogma.ai/2019/08/acting-without-rewards/>.
- [46] Mark Chen, Alec Radford, Rewon Child, Jeffrey Wu, Heewoo Jun, David Luan, and Ilya Sutskever. Generative pretraining from pixels. In *International Conference on Machine Learning*, pages 1691–1703. PMLR, 2020.
- [47] Dibya Ghosh, Abhishek Gupta, Justin Fu, Ashwin Reddy, Coline Devin, Benjamin Eysenbach, and Sergey Levine. Learning to reach goals without reinforcement learning. *arXiv preprint arXiv:1912.06088*, 2019.
- [48] Keiran Paster, Sheila A McIlraith, and Jimmy Ba. Planning from pixels using inverse dynamics models. *arXiv preprint arXiv:2012.02419*, 2020.
- [49] Michael Janner, Qiyang Li, and Sergey Levine. Reinforcement learning as one big sequence modeling problem. *arXiv preprint arXiv:2106.02039*, 2021.
- [50] Johan Ferret, Raphaël Marinier, Matthieu Geist, and Olivier Pietquin. Self-attentional credit assignment for transfer in reinforcement learning. *arXiv preprint arXiv:1907.08027*, 2019.
- [51] Anna Harutyunyan, Will Dabney, Thomas Mesnard, Mohammad Azar, Bilal Piot, Nicolas Heess, Hado van Hasselt, Greg Wayne, Satinder Singh, Doina Precup, et al. Hindsight credit assignment. *arXiv preprint arXiv:1912.02503*, 2019.
- [52] Jose A Arjona-Medina, Michael Gillhofer, Michael Widrich, Thomas Unterthiner, Johannes Brandstetter, and Sepp Hochreiter. Rudder: Return decomposition for delayed rewards. *arXiv preprint arXiv:1806.07857*, 2018.
- [53] Yang Liu, Yunan Luo, Yuanyi Zhong, Xi Chen, Qiang Liu, and Jian Peng. Sequence modeling of temporal credit assignment for episodic reinforcement learning. *arXiv preprint arXiv:1905.13420*, 2019.
- [54] Tero Karras, Samuli Laine, and Timo Aila. A style-based generator architecture for generative adversarial networks. In *Conference on Computer Vision and Pattern Recognition*, 2019.
- [55] Marjan Ghazvininejad, Xing Shi, Jay Priyadarshi, and Kevin Knight. Hafez: an interactive poetry generation system. In *Proceedings of ACL, System Demonstrations*, 2017.

- [56] Lilian Weng. Controllable neural text generation. *lilianweng.github.io/lil-log*, 2021. URL <https://lilianweng.github.io/lil-log/2021/01/02/controllable-neural-text-generation.html>.
- [57] Jessica Fidler and Yoav Goldberg. Controlling linguistic style aspects in neural language generation. *arXiv preprint arXiv:1707.02633*, 2017.
- [58] Zhiting Hu, Zichao Yang, Xiaodan Liang, Ruslan Salakhutdinov, and Eric P Xing. Toward controlled generation of text. In *International Conference on Machine Learning*, 2017.
- [59] Nazneen Fatema Rajani, Bryan McCann, Caiming Xiong, and Richard Socher. Explain yourself! leveraging language models for commonsense reasoning. *arXiv preprint arXiv:1906.02361*, 2019.
- [60] Lantao Yu, Weinan Zhang, Jun Wang, and Yong Yu. Seqgan: Sequence generative adversarial nets with policy gradient. In *AAAI conference on artificial intelligence*, 2017.
- [61] Daniel M Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*, 2019.
- [62] Nitish Shirish Keskar, Bryan McCann, Lav R Varshney, Caiming Xiong, and Richard Socher. Ctrl: A conditional transformer language model for controllable generation. *arXiv preprint arXiv:1909.05858*, 2019.
- [63] Sumanth Dathathri, Andrea Madotto, Janice Lan, Jane Hung, Eric Frank, Piero Molino, Jason Yosinski, and Rosanne Liu. Plug and play language models: A simple approach to controlled text generation. *arXiv preprint arXiv:1912.02164*, 2019.
- [64] Ari Holtzman, Jan Buys, Maxwell Forbes, Antoine Bosselut, David Golub, and Yejin Choi. Learning to write with cooperative discriminators. *arXiv preprint arXiv:1805.06087*, 2018.
- [65] Ben Krause, Akhilesh Deepak Gotmare, Bryan McCann, Nitish Shirish Keskar, Shafiq Joty, Richard Socher, and Nazneen Fatema Rajani. Gedi: Generative discriminator guided sequence generation. *arXiv preprint arXiv:2009.06367*, 2020.
- [66] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- [67] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *European Conference on Computer Vision*, 2020.
- [68] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
- [69] Sam Ritter, Ryan Faulkner, Laurent Sartran, Adam Santoro, Matt Botvinick, and David Raposo. Rapid task-solving in novel environments. *arXiv preprint arXiv:2006.03662*, 2020.
- [70] Sudeep Dasari and Abhinav Gupta. Transformers for one-shot visual imitation. *arXiv preprint arXiv:2011.05970*, 2020.
- [71] Josh Abramson, Arun Ahuja, Iain Barr, Arthur Brussee, Federico Carnevale, Mary Cassin, Rachita Chhaparia, Stephen Clark, Bogdan Damoc, Andrew Dudzik, et al. Imitating interactive intelligence. *arXiv preprint arXiv:2012.05672*, 2020.
- [72] Thomas Wolf, Julien Chaumond, Lysandre Debut, Victor Sanh, Clement Delangue, Anthony Moi, Pierric Cistac, Morgan Funtowicz, Joe Davison, Sam Shleifer, et al. Transformers: State-of-the-art natural language processing. In *Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020.
- [73] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

A 实验细节

实验代码见补充材料。

A.1 雅达利

我们基于 minGPT (<https://github.com/karpathy/minGPT>) 构建了用于 Atari 游戏的决策 Transformer 实现, minGPT 是 GPT 的一个公开可用的重新实现。我们采用了其字符级 GPT 示例中的大部分默认超参数 (https://github.com/karpathy/minGPT/blob/master/play_char.ipynb)。为了加快训练速度, 我们减小了批大小 (Pong 除外)、块大小、层数、注意力头数和嵌入维度。对于观测值的处理, 我们使用 Mnih 等人 [20] 提出的 DQN 编码器, 并添加一个额外的线性层以投影到嵌入维度。

对于回报目标条件化, 我们使用数据集中的最大回报 $1\times$ 或 $5\times$, 但存在更多有原则的回报目标条件化可能性。在 Atari 实验中, 我们在嵌入每种模态后使用双曲正切函数 (Tanh) 而不是层归一化 (LayerNorm) (如第 3 节所述), 但这并未对性能产生显著影响。完整的超参数列表见表 8。

表 8: Atari 实验中 DT (及 %BC) 的超参数。

Hyperparameter	Value
Number of layers	6
Number of attention heads	8
Embedding dimension	128
Batch size	512 Pong 128 Breakout, Qbert, Seaquest
Context length K	50 Pong 30 Breakout, Qbert, Seaquest
Return-to-go conditioning	90 Breakout ($\approx 1\times$ max in dataset) 2500 Qbert ($\approx 5\times$ max in dataset) 20 Pong ($\approx 1\times$ max in dataset) 1450 Seaquest ($\approx 5\times$ max in dataset)
Nonlinearity	ReLU, encoder GeLU, otherwise
Encoder channels	32, 64, 64
Encoder filter sizes	$8\times 8, 4\times 4, 3\times 3$
Encoder strides	4, 2, 1
Max epochs	5
Dropout	0.1
Learning rate	6×10^{-4}
Adam betas	(0.9, 0.95)
Grad norm clip	1.0
Weight decay	0.1
Learning rate decay	Linear warmup and cosine decay (see code for details)
Warmup tokens	512×20
Final tokens	$2\times 500000\times K$

A.2 OpenAI Gym

A.2.1 决策 Transformer

我们的代码基于 Huggingface Transformers 库 [72]。我们在所有 OpenAI Gym 任务上的超参数如下表 9 所示。启发式地, 我们发现与标准强化学习模型规模 (学习单一策略) 相比, 使用更大的模型有助于建模回报分布。对于 reacher 环境, 我们使用比其他环境更短的上下文长度, 我们发现这很有帮助, 因为该环境是目标条件化的, 且回合较短。我们根据每个环境的专家性能选择回报目标, 除了 HalfCheetah, 我们发现 50% 的性能更好, 因为该数据集中的回报相对于其他环境较低。模型使用 AdamW 优化器 [73] 按照 PyTorch 默认设置训练了 10^5 个梯度步骤。

表 9: OpenAI Gym 实验中决策 Transformer 的超参数。

Hyperparameter	Value
Number of layers	3
Number of attention heads	1
Embedding dimension	128
Nonlinearity function	ReLU
Batch size	64
Context length K	20 HalfCheetah, Hopper, Walker 5 Reacher
Return-to-go conditioning	6000 HalfCheetah 3600 Hopper 5000 Walker 50 Reacher
Dropout	0.1
Learning rate	10^{-4}
Grad norm clip	0.25
Weight decay	10^{-4}
Learning rate decay	Linear warmup for first 10^5 training steps

A.2.2 行为克隆

如第 4.2 节简要提及，我们发现先前报告的行为克隆基线较弱，因此采用与决策 Transformer 类似的设置自行运行。我们尝试使用变换器架构，但发现使用多层感知机（如先前工作）效果更强。我们训练 2.5×10^4 个梯度步；更多训练并未提升性能。其他超参数见表 10。百分位行为克隆实验使用相同的超参数。

表 10: OpenAI Gym 实验中行为克隆的超参数。

Hyperparameter	Value
Number of layers	3
Embedding dimension	256
Nonlinearity function	ReLU
Batch size	64
Dropout	0.1
Learning rate	10^{-4}
权重衰减 学习率衰减	10^{-4} 前 10^5 个训练步的线性预热

A.3 图最短路径

我们给出引言中讨论的说明性示例的细节。任务是在固定有向图上寻找最短路径，该问题可形式化为一个马尔可夫决策过程，其中当智能体位于目标节点时奖励为 0，否则为 -1 。观测是智能体所在图节点的整数索引。动作是下一步要移动到的图节点的整数索引。若图中存在边，则转移动态将智能体传送到动作对应的节点索引，否则智能体停留在原节点。该问题中的回报到目标对应负路径长度，最大化它们对应生成最短路径。

在此环境中，我们使用第 3 节描述的 GPT 模型同时生成动作和回报到目标标记。这使得模型能够生成自身（可实现的）回报到目标 $\hat{R}$ 。由于生成动作需要回报提示，且我们假设预先知道最优路径长度，因此使用一个偏向较短路径的简单回报先验： $P_{\text{prior}}(\hat{R} = k) \propto T + 1 - k$ ，其中 T 是最大轨迹长度。然后，将其与 GPT 模型生成的回报概率结合： $P(\hat{R}_t | s_{0:t}, a_{0:t-1}, \hat{R}_{0:t-1}) =$



图 6: 图上随机游走、到达目标节点的最短可能路径以及变换器模型生成的尝试最短路径的步数直方图。 ∞ 表示轨迹期间未到达目标。

$P_{\text{GPT}}(\hat{R}_t | s_{0:t}, a_{0:t-1}, \hat{R}_{0:t-1}) \times P_{\text{prior}}(\hat{R}_t)^{10}$ 。注意，先验和回报到目标预测完全可由模型计算，因此避免了任何外部或预言信息（如最优路径长度）的需求。通过先验调整生成在先前可控文本生成工作中也用于类似目的 [65]。

我们在包含 1,000 条图随机游走轨迹的数据集上进行训练，每条轨迹包含 $T = 10$ 步，使用具有 20 个节点且边稀疏系数为 0.1 的随机图。我们在图 6 中报告了结果，发现变换器模型能够显著减少到达目标所需的步数，其性能与最优路径非常接近。

该任务上性能优异的原因有两个。一种情况是，随机游走轨迹的训练数据集中可能包含直接对应于期望最短路径的片段，此时模型会生成该片段。第二种情况是，生成的路径完全是原创的，并非训练数据集中轨迹的子集——它们是通过拼接次优片段生成的。我们发现这种情况在实验中占生成路径的 15.8%。

虽然这是一个简单的示例，并且为了简化在其他实验中未使用生成先验，但它说明了如何将事后回报信息与生成先验结合使用，以避免显式的动态规划。

B Atari 任务得分

表 11 展示了 Hafner 等人 [21] 中用于归一化的标准化得分。表 12 和表 13 分别展示了与表 1 和表 4 对应的原始得分。对于 %BC 得分，我们使用与决策 Transformer 相同的超参数以进行公平比较。对于 REM 和 QR-DQN，Agarwal 等人 [13] 与 Kumar 等人 [14] 之间存在轻微差异；我们报告 REM 作者提供给我们的原始数据。

Game	Random	Gamer
Breakout	2	30
Qbert	164	13455
Pong	-21	15
Seaquest	68	42055

表 11: 用于归一化的 Atari 基线得分。

Game	DT (Ours)	CQL	QR-DQN	REM	BC
Breakout	76.9 $\pm$ 27.3	61.1	6.8	4.5	40.9 $\pm$ 17.3
Qbert	2215.8 $\pm$ 1523.7	14012.0	156.0	160.1	2464.1 $\pm$ 1948.2
Pong	17.1 $\pm$ 2.9	19.3	-14.5	-20.8	9.7 $\pm$ 7.2
Seaquest	1129.3 $\pm$ 189.0	779.4	250.1	370.5	968.6 $\pm$ 133.8

表 12: 1% DQN-replay Atari 数据集的原始得分。我们报告了 3 个随机种子下的均值和方差。最佳均值以粗体突出显示。决策 Transformer 在 4 个游戏中的 3 个上与 CQL 表现相当，并且通常优于其他基线。

Game	DT (Ours)	10%BC	25%BC	40%BC	100%BC
Breakout	76.9 $\pm$ 27.3	10.0 $\pm$ 2.3	22.6 $\pm$ 1.8	32.3 $\pm$ 18.9	40.9 $\pm$ 17.3
Qbert	2215.8 $\pm$ 1523.7	1045 $\pm$ 232.0	2302.5 $\pm$ 1844.1	1674.1 $\pm$ 776.0	2464.1 $\pm$ 1948.2
Pong	17.1 $\pm$ 2.9	-20.3 $\pm$ 0.1	-16.2 $\pm$ 1.0	5.2 $\pm$ 4.8	9.7 $\pm$ 7.2
Seaquest	1129.3 $\pm$ 189.0	521.3 $\pm$ 103.0	549.3 $\pm$ 96.2	758 $\pm$ 169.1	968.6 $\pm$ 133.8

表 13: Atari 的 %BC 得分。我们报告了 3 个随机种子下的均值和方差。决策 Transformer 通常优于 %BC。